

PROJECT DOCUMENTATION

Multi-Line Insurance Policy and Claims Management System

Salesforce implementation: Auto, Property and Life insurance

Platform	Salesforce (Lightning Experience)
Insurance lines	Auto (vehicle), Property, Life
Key components	Custom objects, record types, Screen Flows, Record-Triggered Flows, Apex, Lightning Web Components, Approval Process, permission sets
Milestones covered	1. Data model and policy configuration 2. Policy issuance and claim routing automation 3. Claims adjuster LWC dashboard 4. Advanced claim processing, security and code setup

Table of Contents

1. Project Overview	3
1.1 Introduction	3
1.2 Problem Statement	3
1.3 Business Goals	3
2. Objectives and Scope	3
2.1 Project Objectives	3
2.2 Scope and Deliverables by Milestone	3
3. Challenges and Solution Approach	4
3.1 Key Challenges	4
3.2 Solution Approach	4
4. System Architecture	4
4.1 Three-Tier Architecture on Salesforce	4
4.2 Tools and Technologies Used	5
5. User Roles and Responsibilities	5
6. Functional Modules	6
7. Data Model	7
7.1 Policy Object (Policy__c)	7
7.2 Record Types and Field Sets	7
7.3 Claim Object (Claim__c)	8
7.4 Entity Relationships	8
8. Milestone 1: Core Data Model and Policy Configuration	9
8.1 AutoQuotingFlow (Screen Flow)	9
8.2 Validation Rule: VIN Must Be 17 Characters	10
9. Milestone 2: Policy Issuance and Claim Routing Automation	10
9.1 PremiumCalculator (Apex)	10
9.2 Integrating the Apex Action into AutoQuotingFlow	11
9.3 ClaimRoutingFlow (Record-Triggered, After Save)	11
10. Milestone 3: Claims Adjuster LWC Dashboard	12
10.1 ClaimsAdjusterController (Apex)	12
10.2 claimsDashboardLwc	12
10.3 claimTileLwc	12
11. Milestone 4: Advanced Claim Processing, Security and Code Setup	13
11.1 High Value Claim Approval Process	13
11.2 Submission Automation Flow	14
11.3 Claim Approver Screen Flow and Quick Action	14
11.4 Territory-Based Claim Visibility	15
11.5 Permission Sets	16
11.6 Apex Test Class: ClaimsAdjusterControllerTest	16
12. Security Summary	17
13. Testing and Validation Scenarios	17
14. Observations and Recommendations	18
15. Conclusion and Future Enhancements	19
15.1 Conclusion	19
15.2 Future Enhancements	19

1. Project Overview

1.1 Introduction

The **Multi-Line Insurance Policy and Claims Management System** is a Salesforce-based solution that manages Auto (vehicle), Property and Life insurance on a single platform. It covers the complete insurance workflow: quoting, policy issuance, claim intake, automatic claim routing, multi-level approval, adjuster workload management and role-based data security.

This document combines two project inputs: the **conceptual project overview** (objectives, user roles, modules, data structure and security features) and the **Salesforce implementation guide** (four milestones with step-by-step build instructions). The conceptual three-tier architecture is retained and mapped to Salesforce components, so the reader can see how each business requirement is realised in the platform.

1.2 Problem Statement

An insurance carrier that sells vehicle, property and life insurance currently runs a legacy system. Policy quoting, issuance and claims handling are manual, slow and inconsistent. There is no unified platform, so cross-selling is difficult. The manual claims review process causes long resolution times, high operational cost and a poor customer experience.

1.3 Business Goals

- **Standardise and accelerate** quoting and policy issuance for all insurance lines.
- **Automate claim initiation and routing** based on policy type and claim severity (amount).
- **Provide a 360-degree view** of Customer, Policy and Claim data in one place.
- Reduce paperwork and manual processing, and improve the overall efficiency of insurance operations.

Implementation platform

The conceptual overview lists a web stack (React, FastAPI/Flask, MySQL/PostgreSQL). The delivered solution is implemented on **Salesforce** (custom objects, Flows, Apex, LWC, Approval Processes). Both views are kept in this document: the concept in Section 4 and the real build in Sections 7 to 11.

2. Objectives and Scope

2.1 Project Objectives

- Manage multiple insurance lines (Auto, Property, Life) from a single system using a flexible data model.
- Guide agents through multi-step quoting using **Screen Flows**, with minimum clicks.
- Let customers and agents view and manage policies, and let claims be created and tracked with a clear status.
- Build a dynamic **Apex/LWC Claims Dashboard** so adjusters can manage their workload efficiently.
- Automate claim routing and approval/escalation using **Record-Triggered Flows** and an **Approval Process**.
- Enforce data validation (for example VIN length) to keep data compliant and accurate.
- Protect data with role-based access, sharing rules and permission sets.

2.2 Scope and Deliverables by Milestone

Milestone	Focus	Key deliverables
Milestone 1	Core data model and policy configuration	Policy and Claim objects, record types, field sets, custom fields, AutoQuotingFlow (Screen Flow), VIN validation rule
Milestone 2	Policy issuance and claim routing automation	PremiumCalculator Apex class, premium update in AutoQuotingFlow, ClaimRoutingFlow (queue assignment by policy type)
Milestone 3	Claims adjuster LWC dashboard	ClaimsAdjusterController (Apex), claimsDashboardLwc, claimTileLwc
Milestone 4	Advanced claim processing, security and code setup	High Value Claim Approval Process, submission flow, approver Screen Flow and quick action, test class, state-based sharing, permission sets

3. Challenges and Solution Approach

3.1 Key Challenges

Challenge	Why it matters	How the solution addresses it
Variable data management	Vehicle needs VIN and model year; Property needs square footage and year built; Life needs beneficiary and term.	One Policy object with three Record Types and three Field Sets (Vehicle, Property, Life).
Approval complexity	Approval depends on claim amount and approver level.	Entry criterion of claim amount above 50,000 and a two-step approval (Senior Adjuster, then Claims Manager).
API integration simulation	Quote verification against an external vehicle/property database needs a callout and mock tests.	Rating logic is built in Apex (PremiumCalculator). A callout with a mock test class is listed as a future enhancement (Section 15).
User adoption	Agents need fewer clicks during quoting.	Short Screen Flow: basic information screen, vehicle screen, then automatic draft policy and premium.

3.2 Solution Approach

- **Flexible data model:** Policy and Claim objects with Record Types for Vehicle, Property and Life.
- **Declarative quoting:** multi-screen Screen Flows capture policy-specific data.
- **Programmatic and flow-based claims logic:** Apex for rating, Record-Triggered Flows for routing and approval submission.
- **Focused UI:** a Lightning Web Component dashboard for adjusters with client-side filtering.

4. System Architecture

4.1 Three-Tier Architecture on Salesforce

The system follows a three-tier architecture: presentation, application and data. In the conceptual design these tiers are HTML/CSS/JS (React), a REST backend and a database. On Salesforce, the same tiers are provided by the platform.

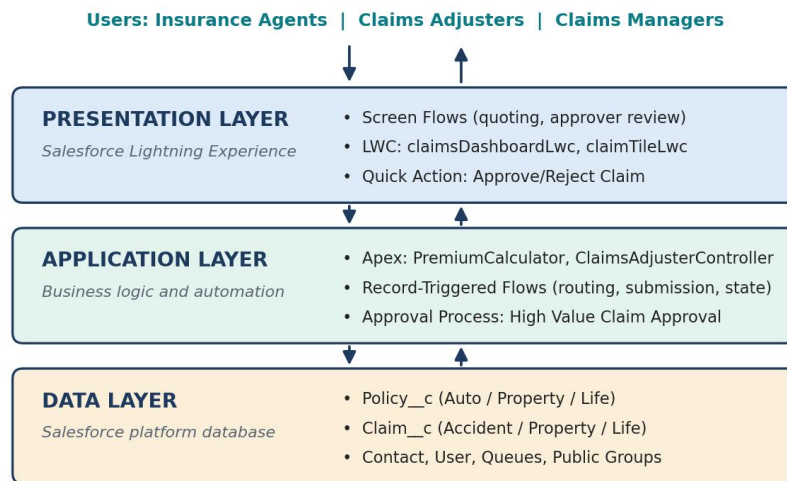


Figure 4.1: Three-tier architecture mapped to Salesforce components

Layer	Conceptual design	Salesforce implementation
Presentation	HTML, CSS, JavaScript, React front end	Lightning Experience, Screen Flows, LWC dashboard, quick actions
Application	Backend REST API: authentication, policy, claims, user and admin management	Apex classes, Record-Triggered Flows, Approval Process, permission sets
Data	Users, Policies, Claims, Payments, Insurance details, Documents	Contact, User, Policy__c, Claim__c, queues and public groups

4.2 Tools and Technologies Used

Tool / Technology	Usage in the project
Setup and Object Manager	Create custom objects, record types, field sets, fields, validation rules, page layouts
Flow Builder	AutoQuotingFlow, ClaimRoutingFlow, Submission Automation Flow, Claim Approver Screen Flow, Claim Policy Holder State Update
Developer Console and Apex	PremiumCalculator, ClaimsAdjusterController, ClaimsAdjusterControllerTest
Lightning Web Components	claimsDashboardLwc and claimTileLwc, deployed using the Lightning Studio Chrome extension
Approval Processes	High Value Claim Approval with two sequential steps
Sharing Settings, Permission Sets	State-based claim sharing and role-based access (Agent, Adjuster, Manager)

5. User Roles and Responsibilities

The conceptual design defines two role groups, Customer and Administrator. In the Salesforce build these are refined into four working roles, each with its own permission set.

Role	Responsibilities	Access
Customer / Policy Holder	Registers, views profile and policies, buys or manages policies, submits claims with documents, tracks claim status and history. In this build the customer is stored as a Contact and served through agents.	Contact record linked to Policy__c through the Customer lookup
Insurance Agent	Quoting only: creates quotes and draft policies using AutoQuotingFlow, maintains customer contact data.	Permission set: Insurance Agent Access
Claims Adjuster	Claim handling: works on claims assigned to them, uses the dashboard, updates claim details.	Permission set: Claims Adjuster Access; state-based sharing
Claims Manager (and Senior Adjuster)	Reporting and approvals: reviews high-value claims, approves or rejects, runs reports.	Permission set: Claims Manager Access

Administrator

Administrators configure objects, flows, queues, groups and permission sets, and manage insurance records and system reports. They also keep the approval process and sharing rules aligned with business rules.

6. Functional Modules

The seven modules of the conceptual design are mapped below to the components that implement them.

Module	What it handles	Salesforce implementation
1. Authentication	Registration, login, logout, role-based access	Salesforce login, profiles, permission sets
2. Customer management	Customer profile, contact information, policy and claim history	Contact records; Customer lookup on Policy__c
3. Policy management	Policy creation, type, coverage, premium, start date, status	Policy__c, AutoQuotingFlow, VIN validation rule, PremiumCalculator
4. Insurance product management	Multiple insurance lines with different requirements	Record Types and Field Sets: Auto, Property, Life
5. Claims management	Claim submission, number generation, verification, approval or rejection, status tracking	Claim__c with auto number C-{0000}, ClaimRoutingFlow, Approval Process, Approval Status field
6. Dashboard	Pending, approved and rejected claims, workload view	claimsDashboardLwc and claimTileLwc for adjusters
7. Reports and analytics	Policy and claim statistics, approval trends	Reports enabled on both objects; Run Reports permission for managers

The conceptual design also lists **Payments** and **Documents** (claim document upload). These are not part of the supplied build steps and are included as future enhancements.

7. Data Model

7.1 Policy Object (Policy__c)

The Policy custom object stores the policy for every insurance line. The record name is an auto number, so every policy gets a unique, readable identifier.

Setting	Value
Label / Plural label	Policy / Policies
Record name	Policy Number (Auto Number), format P-{0000}, starting number 1
Optional features	Allow Reports, Allow Activities, Track Field History
Deployment and search	Deployed; Allow Search enabled

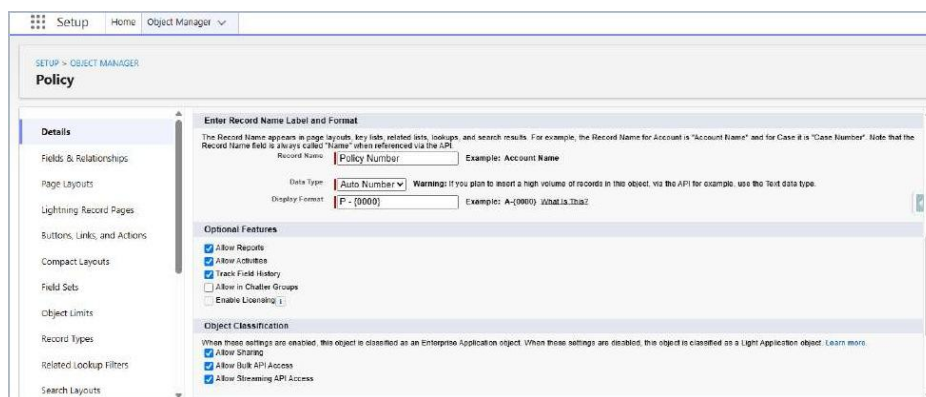


Figure 7.1: Policy object, record name and optional features

Policy fields

No.	Field	Data type	Used for
1	Beneficiary Name	Text (50)	Life
2	Customer	Lookup (Contact)	All lines (policy holder)
3	Model Year	Text (5)	Auto
4	Policy Start Date	Date	All lines
5	Policy Term Months	Number (18, 0)	Life
6	Premium	Currency (16, 2)	All lines (calculated by Apex)
7	Square Footage	Number (18, 0)	Property
8	VIN	Text (20)	Auto
9	Year Built	Text (5)	Property

The flows and Apex also use a **Policy State** picklist on Policy__c, which drives the state-based premium rating.

7.2 Record Types and Field Sets

Each insurance line is a Record Type of the Policy object (created from the Master record type, active, available to all profiles). A Field Set per line groups the fields that belong to that line, so screens and layouts can show only relevant data.

Line	Record type	Field set	Fields in the field set
Vehicle	Auto	Vehicle	VIN, Model Year
Property	Property	Property	Square Footage, Year Built
Life	Life	Life	Beneficiary Name, Policy Term Months

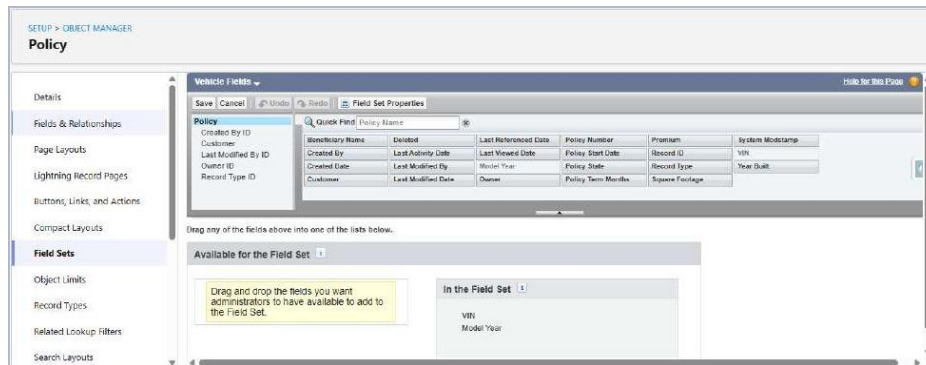


Figure 7.2: Vehicle field set containing VIN and Model Year

7.3 Claim Object (Claim__c)

The Claim object is created the same way as Policy: record name **Claim Number**, auto number format **C-{0000}**, starting number 1, with Reports, Activities and Field History tracking enabled. Its Record Types are **Accident**, **Property** and **Life**.

No.	Field	Data type	Purpose
1	Adjuster	Lookup (User)	Assigned adjuster
2	Approval Status	Picklist: New, Submitted for Approval, Approved, Rejected	Claim lifecycle status
3	Claim Amount	Currency (16, 2)	Drives approval entry criteria
4	Date of Loss	Date/Time	Used to calculate days open
5	Description	Long Text Area	Claim details
6	Policy	Lookup (Policy)	Parent policy

Milestone 4 also uses **Policy Account Holder State** on Claim, filled by a flow and used by sharing rules (Section 11.4).

7.4 Entity Relationships

- One Contact (customer) can hold many Policies; one Policy can have many Claims.
- Each Claim has an owner (queue or adjuster) and an optional Adjuster lookup.
- Claims above 50,000 enter the High Value Claim Approval process.

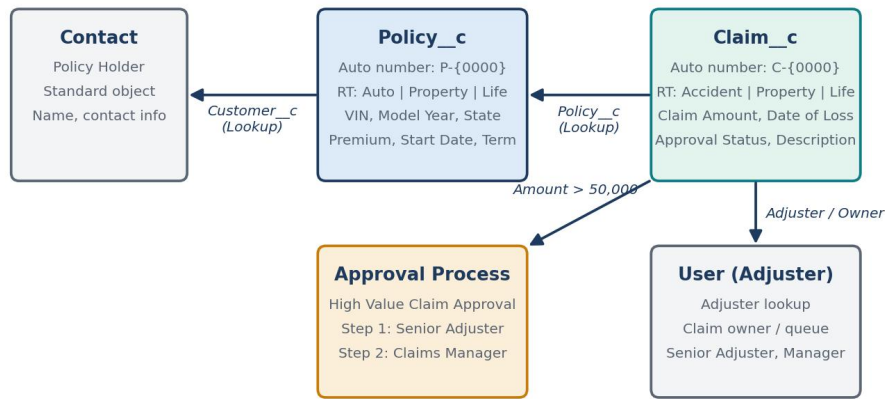


Figure 7.3: Relationships between Contact, Policy, Claim, User and the approval process

8. Milestone 1: Core Data Model and Policy Configuration

Milestone 1 builds the foundation: the Policy and Claim objects (Section 7), the quoting Screen Flow and the VIN validation rule.

8.1 AutoQuotingFlow (Screen Flow)

The flow lets an agent create an Auto policy quote in two screens. The elements are listed below in execution order.

#	Element	Type	Configuration
1	Get Vehicle RT ID	Get Records	Object: Record Type. Conditions: SObject Type Name = Policy__c AND Record Type Name = Auto. Stores the first record.
2	New Auto Policy Quote - Basic Information	Screen	Display text (Screen_1_Policy_Basics); Lookup Policy Holder (Customer__c on Policy__c); Date field Policy Start Date; Picklist Policy State (picklist choice set from Policy State field).
3	Vehicle-Specific Details	Screen	Display text (Screen_2_Vehicle_Details); text inputs VIN and Model Year.
4	Create Draft Policy	Create Records	Maps Customer, Model Year, Policy Start Date, Policy State, Record Type ID and VIN. Policy ID stored in variable varPolicyId (Text).

Two more elements, **Calculate Premium Action** and **Update Policy With Premium**, are added in Milestone 2 (Section 9.2).



A validation rule on Policy enforces a valid VIN for Auto policies. The rule is active, the error is shown on the VIN field.



PremiumCalculator exposes an **invocable method** (`Calculate Policy Premium`) so a flow can call it. The input wrapper carries the `policyId`. The method queries VIN, Model Year and Policy State for the given policies and returns a premium for each.

Rating rules

Rule	Effect
Base premium	1,000.00
Policy State = CA	Base premium multiplied by 1.15
Policy State = TX	Base premium multiplied by 1.05
Model Year earlier than 2018	Add 100 to the premium

Worked examples

State	Model year	Calculation	Premium
CA	2020	1000 x 1.15	1,150.00
TX	2015	1000 x 1.05 + 100	1,150.00
CA	2015	1000 x 1.15 + 100	1,250.00
Any other state	2022	1000	1,000.00

9.2 Integrating the Apex Action into AutoQuotingFlow

#	Element	Type	Configuration
1	Calculate Premium Action	Apex Action	Action: Calculate Policy Premium. Input policyId = varPolicyId. Output stored manually in new variable varCalculatedPremium (Currency, 2 decimal places).
2	Update Policy With Premium	Update Records	Condition: Record Id = varPolicyId. Field value: Premium = varCalculatedPremium.

After adding both elements, the flow is saved as a new version and activated.

9.3 ClaimRoutingFlow (Record-Triggered, After Save)

This flow assigns every new claim to the correct queue, based on the Record Type of its policy.

#	Element	Type	Configuration
1	Start	Trigger	Object: Claim. Trigger: record is created. Optimised for Actions and Related Records.
2	Get_Policy_RT	Get Records	Policy where Record Id = Triggering Claim > Policy. First record only; Record Type ID stored in varPolicyRTName.
3	Route by Policy Type	Decision	Outcome Auto Claim Route (varPolicyRTName = Auto); Property Claim Route (= Property); default outcome Life Claim Route.
4	Assign to Auto / Property / Life Queue	Update Records	Updates the triggering Claim and sets Owner ID to the matching queue.

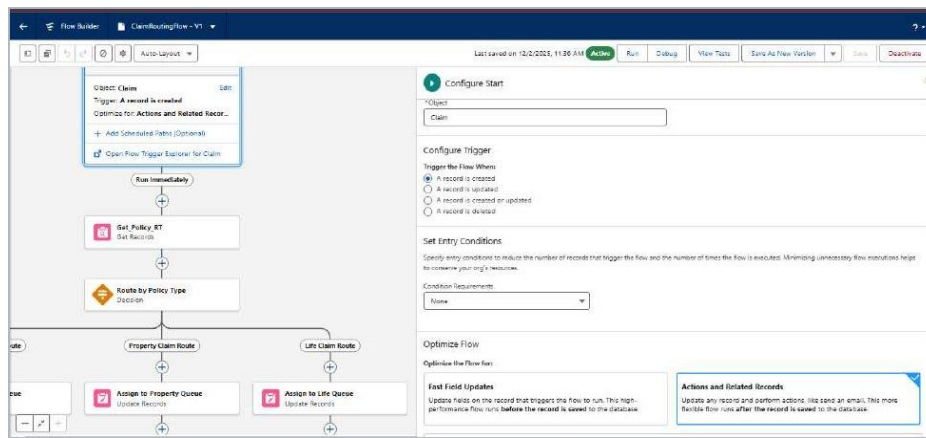


Figure 9.1: ClaimRoutingFlow with the Route by Policy Type decision

10. Milestone 3: Claims Adjuster LWC Dashboard

Milestone 3 gives each adjuster a focused dashboard of the claims they own. It uses one Apex controller and two Lightning Web Components (a parent dashboard and a reusable tile).

10.1 ClaimsAdjusterController (Apex)

The controller returns a list of **ClaimWrapper** objects. The method `getAssignedClaims()` is `@AuraEnabled(cacheable=true)` and uses SOQL to read all claims where the owner is the logged-in user, including related Policy and Customer data, sorted by Date of Loss (latest first).

Wrapper field	Type	Source
claimId	Id	Claim Id
claimNumber	String	Claim Name (auto number)
claimAmount	Decimal	Claim_Amount__c
status	String	Approval_Status__c
policyType	String	Policy record type (Unknown if not available)
policyHolderName	String	Policy Customer (Contact) name
daysOpen	Integer	Calculated from Date_of_Loss__c

10.2 claimsDashboardLwc

- Wires the Apex method with `@wire(getAssignedClaims)` and keeps two lists: **allClaims** (full data) and **visibleClaims** (filtered data).
- Shows a **Filter by Policy Type** combobox with All, Auto, Property and Life. The filter runs on the client side, so no extra server call is needed.
- Displays one **c-claim-tile-lwc** per visible claim in a responsive grid (1, 2 or 3 columns by screen size) and a **Total Claims Assigned** count.
- Exposed to **Lightning App Page** and **Home Page**.

10.3 claimTileLwc

A reusable child component that receives a claim through the `@api claimData` property and shows its essential information.

Item	Behaviour
Header	Claim number and policy type with an icon
Body	Policy holder, claim amount (currency format, USD), status and days open
Icon: Auto	utility:truck
Icon: Property	utility:home
Icon: Life	utility:connected_apps
Icon: unknown type	utility:help

Deployment

Both components are created and deployed from the **Lightning Studio** Chrome extension: Create New, Lightning Web Component, enter the name, tick isExposed, choose the targets and click Deploy. Then the HTML, JS and meta XML are pasted into the three files.

11. Milestone 4: Advanced Claim Processing, Security and Code Setup

11.1 High Value Claim Approval Process

A standard approval process on Claim sends large claims through two sequential approval steps. The Approval Status picklist (New, Submitted for Approval, Approved, Rejected) tracks progress.

Setting	Value
Process name	High Value Claim Approval (standard setup wizard)
Entry criteria	Claim: Claim Amount greater than 50000
Record editability	Administrators only can edit records during approval
Approval page layout fields	Claim Number, Owner, Claim Amount, Adjuster, Policy
Security setting	Approvers access the page only from within Salesforce
Initial submitters	Claim Owner

The screenshot shows the Salesforce Setup interface for configuring an Approval Process. The left sidebar contains navigation links: Setup, Home, Object Manager, and a search bar. Under 'Setup', there are links for Data, Feature Settings, and Process Automation. The 'Approval Processes' link is highlighted. The main content area shows the 'Specify Entry Criteria' step of the 'High Value Claim Approval' process. The criteria is defined as 'Claim: Claim Amount greater than 50000'. The interface includes a table for specifying criteria with columns for Field, Operator, and Value. The 'Field' is 'Claim: Claim Amount', the 'Operator' is 'greater than', and the 'Value' is '50000'. There are also buttons for 'Previous', 'Save', 'Next', and 'Cancel'.

Figure 11.1: Approval process entry criteria (Claim Amount greater than 50000)

Approval steps and actions

Item	Configuration	Detail
Step 1: Senior Adjuster	All records enter the step	Auto-assigned to the Senior Adjuster user; approve or reject on the first response
Step 2: Manager Review	All records enter the step	Auto-assigned to the Claims Manager user; on rejection perform all rejection actions for the step and all final rejection actions
Initial submission action	Field update: Update Status Submitted	Approval Status = Submitted for Approval
Final approval action	Field update: Update Status Approved	Approval Status = Approved
Final rejection action	Field update: Update Status Rejected	Approval Status = Rejected

The approval process is then activated.

11.2 Submission Automation Flow

This Record-Triggered Flow submits qualifying claims to the approval process automatically, so adjusters do not need to click Submit for Approval.

Setting	Value
Object and trigger	Claim; record is created or updated
Entry conditions (all)	Claim Amount greater than 50000 AND Approval Status equals New
When to run on update	Every time a record is updated and meets the condition requirements
Optimised for	Actions and Related Records
Action: Submit Claim to Approval	Record Id = Triggering Claim > Record ID; Approval Process = High Value Claim Approval

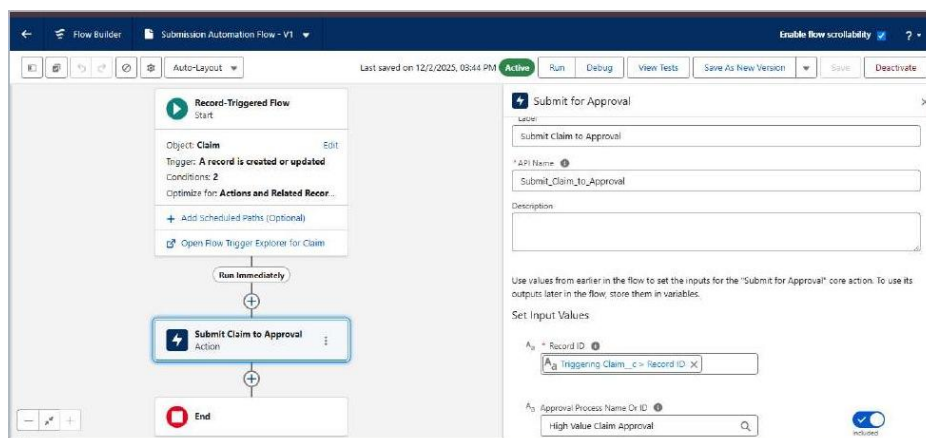


Figure 11.2: Submission Automation Flow with the Submit Claim to Approval action

11.3 Claim Approver Screen Flow and Quick Action

A Screen Flow lets approvers review a claim and record a decision directly from the claim page.

#	Element	Type	Configuration
1	recordId	Variable	Text, available for input (receives the record Id from the quick action)
2	Get Claim Details	Get Records	Claim where Record Id = recordId; first record, all fields
3	Claim Review Screen	Screen	Display text with Claim ID, Amount and Policy Type; radio buttons Approval Decision (Approve / Reject); long text Approver Comments
4	Check Decision	Decision	Decision Approved if the radio value equals Approve; default outcome Decision Rejected
5	Approve Claim / Reject Claim	Action	Submit for Approval action with Record ID = recordId and submission comments = Approver Comments

The flow is saved and activated as **Claim Approver Screen Flow**. A **Flow quick action** named **Approve/Reject Claim** is created on Claim and added to the Salesforce Mobile and Lightning Experience actions of the Claim page layout.

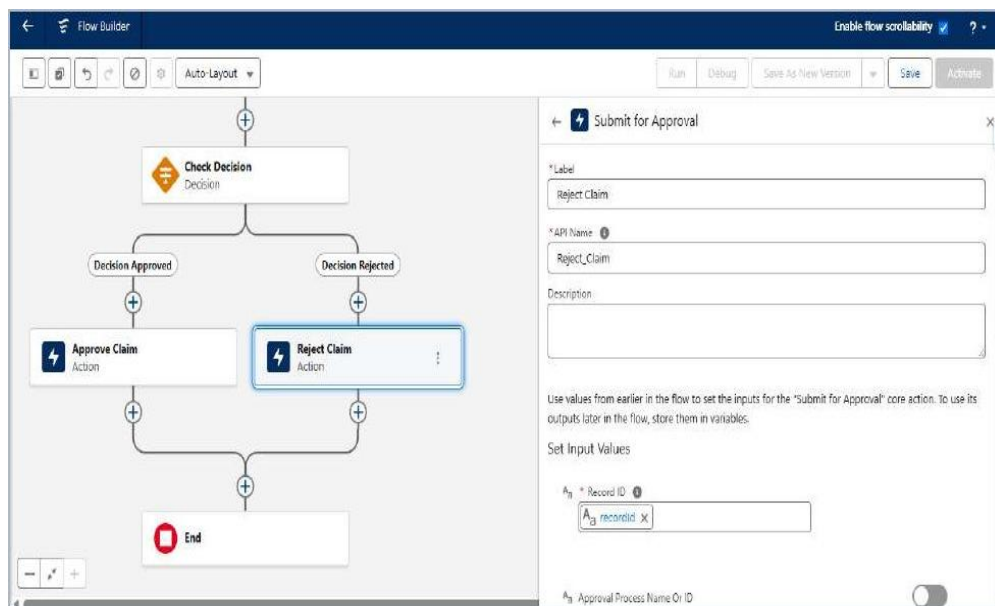


Figure 11.3: Claim Approver Screen Flow with Approve Claim and Reject Claim paths

11.4 Territory-Based Claim Visibility

To make adjusters see claims only in their assigned state, claims first receive the state of their policy, and then criteria-based sharing rules share them with state-specific groups.

Claim Policy Holder State Update (Record-Triggered Flow)

- Object Claim; trigger when a record is created or updated; no entry conditions; optimised for Fast Field Updates.
- `get_policy_records` (Get Records on Policy) stores the Policy State field.
- Loop `Claim_Filed_update` iterates the policies; `Claim_Filed_state_update` sets the claim field **Policy Account Holder State** from the current loop item.

Sharing rules on Claim

Rule	Criteria	Shared with / access
Share Records based on Policy State CA	Policy Account Holder State equals CA	Group: Claim Adjusters - California; Read/Write
Share Records based on Policy State (TX)	Policy Account Holder State equals TX	Group: Claim Adjusters - Texas; Read/Write

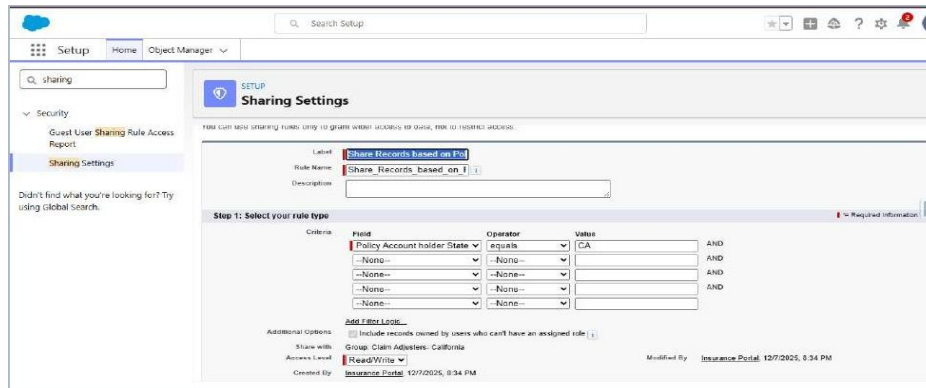


Figure 11.4: Criteria-based sharing rule for state CA (Read/Write)

11.5 Permission Sets

Three permission sets separate quoting, claim handling and management responsibilities.

Permission set	Object access	Other permissions
Insurance Agent Access (quoting only)	Policy: Create, Read Contact: Read, Edit Claim: Read	System: Flow User enabled Apex class: PremiumCalculator
Claims Manager Access (reporting and approvals)	Policy: Read Contact: Read, Edit Claim: Read, Edit, Delete	System: Run Reports, Manage Approvals
Claims Adjuster Access (claim handling)	Policy: Read Claim: Read, Edit	Apex class: ClaimsAdjusterController

11.6 Apex Test Class: ClaimsAdjusterControllerTest

The test class uses a `@testSetup` method to create a Standard User, a Contact (John Doe), an Auto Policy (with a 17-character VIN) and an Accident Claim of 5,000 with Date of Loss 10 days ago, all owned by the test user. The test method then runs as that user and calls `getAssignedClaims()`.

Assertion	Expected result
claimId, claimNumber	Not null
status	New
claimAmount	5000
policyHolderName	Contains John
policyType	Not equal to Unknown

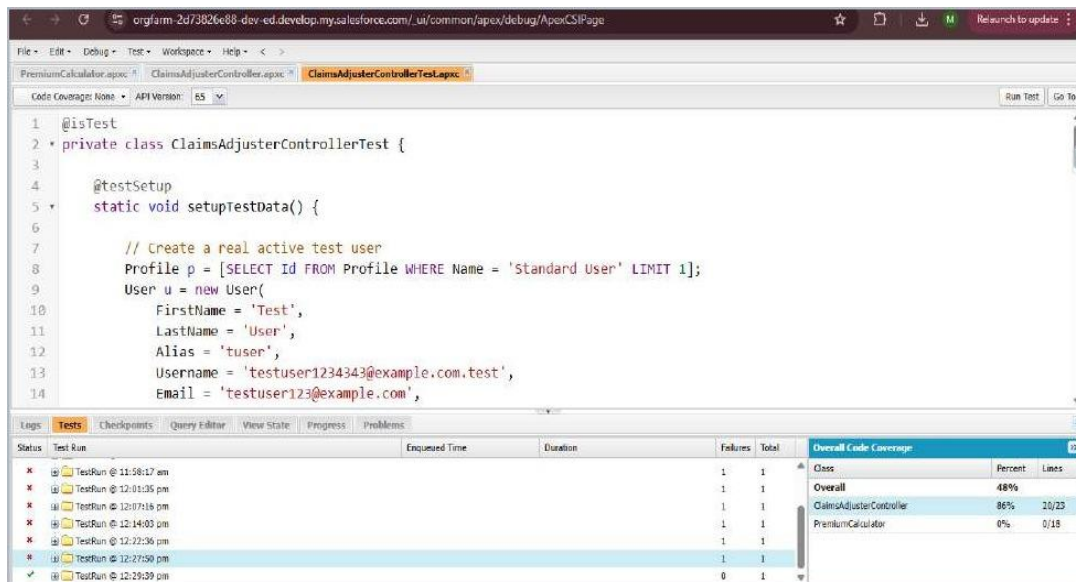


Figure 11.5: Developer Console test run and code coverage result

Class	Coverage	Lines covered
ClaimsAdjusterController	86%	20 of 23
PremiumCalculator	0%	0 of 19 (no test class yet)
Overall (at time of screenshot)	48%	-

12. Security Summary

The security features from the conceptual overview are delivered as follows.

Security feature	How it is delivered
Secure authentication	Salesforce user login and session security; approval pages open only within Salesforce
Role-based authorisation	Three permission sets (Agent, Adjuster, Manager) with object-level permissions
Protected admin functions	Approval records are editable by administrators only during approval; Manage Approvals and Run Reports only for managers
Input and data validation	VIN validation rule; required types and lengths on fields (for example Text(20), Currency(16,2))
Controlled record access	Criteria-based sharing rules by state; adjuster dashboard reads only claims owned by the logged-in user
Audit trail	Field history tracking enabled on Policy and Claim

13. Testing and Validation Scenarios

The following end-to-end scenarios verify that all four milestones work together.

#	Scenario	Steps	Expected result
1	Create Auto quote	Run AutoQuotingFlow; choose customer, start date, state CA; enter VIN (17 characters) and model year 2020	Draft Auto policy is created with Premium 1,150.00
2	Invalid VIN	Enter a VIN with fewer than 17 characters	Validation error shown on the VIN field
3	Claim routing	Create a claim on an Auto, a Property and a Life policy	Owner is set to the Auto, Property or Life queue
4	High-value claim	Create a claim with amount 60,000 and status New	Claim is submitted to High Value Claim Approval (Senior Adjuster, then Manager); status becomes Submitted for Approval, then Approved or Rejected
5	Adjuster dashboard	Log in as an adjuster; open the dashboard; filter by policy type	Only owned claims appear; the list filters instantly
6	State visibility	Create claims for CA and TX policies	Each state group gets Read/Write access to its own claims

14. Observations and Recommendations

While analysing the build steps and screenshots, the following points were noted. They do not stop the project, but resolving them will make it more robust and production-ready.

Area	Observation	Recommendation
Apex controller	The SOQL in ClaimsAdjusterController selects <code>Policy__r.RecordType.DeveloperName</code> , but the wrapper reads <code>RecordType.Name</code> and <code>Customer.Name</code> while only first and last name are queried. This can cause a field-not-queried error or Unknown policy type.	Query exactly the fields that are read (for example <code>RecordType.Name</code> , <code>Customer__r.Name</code>) and re-run the test.
Days open	<code>Date.today().daysBetween(dateOfLoss)</code> returns a negative number for past dates.	Use <code>dateOfLoss.date().daysBetween(Date.today())</code> .
Test coverage	Controller coverage is 86% (target 95%), the key assertion for results is commented out, and PremiumCalculator has 0% coverage.	Add tests for empty results, missing policy or customer and all PremiumCalculator branches; Salesforce also requires 75% overall coverage for deployment.
Hard-coded queue IDs	ClaimRoutingFlow stores 18-digit queue Ids directly.	Look up queues with Get Records (Group, Type = Queue, by developer name) so the flow works in every sandbox and production.
State update flow	<code>get_policy_records</code> has no filter, so the loop reads all policies and the last one can overwrite the claim state.	Filter Policy by Id = Triggering Claim > Policy, or read the state through the Policy relationship.

Area	Observation	Recommendation
Approver Screen Flow	Both Approve and Reject paths use the Submit for Approval action, which starts an approval request; it does not record an approve or reject decision.	Record the decision with an Apex invocable (ProcessWorkitemRequest) or let approvers act on the approval work item.
Sharing rules	Sharing rules only add access. Adjusters are restricted only if the Claim organisation-wide default is Private.	Set Claim OWD to Private and confirm group membership for each state.
PremiumCalculator	Results are returned in query order, not input order, and VIN is queried but unused.	Return results mapped by policy Id order; remove unused fields or use VIN for validation.
Dashboard empty state	An empty array is truthy in <code>if:true</code> , so the No claims message will not appear.	Use a getter such as <code>hasClaims</code> that checks <code>visibleClaims.length</code> .
Duplicate step	The Approval Status field is created in Milestone 1 (Claim fields) and again in Milestone 4.	Create it once; in Milestone 4 only verify the picklist values.

15. Conclusion and Future Enhancements

15.1 Conclusion

The Multi-Line Insurance Policy and Claims Management System replaces manual, inconsistent processes with a unified Salesforce platform. One flexible data model supports three insurance lines. Screen Flows speed up quoting, Apex calculates premiums, Record-Triggered Flows route and escalate claims, an LWC dashboard organises the adjuster workload, and permission sets with sharing rules protect the data. Together these meet the three business goals: faster standard quoting, automated claim routing, and a 360-degree view of customer, policy and claim.

15.2 Future Enhancements

- **External verification:** an Apex callout to a motor vehicle or property database for quote verification, with an `HttpCalloutMock` test class.
- **Customer self-service:** an Experience Cloud portal for registration, policy purchase, claim submission, document upload and status tracking.
- **Claim documents and payments:** Files on the Claim object and a Payment object, as in the conceptual data structure.
- **Reports and dashboards:** claim approval and rejection trends, insurance category performance, pending claims for managers.
- **Smarter approval:** routing by policy type, claim amount, policy history and adjuster level, as in the original requirement.
- **Quoting for Property and Life:** dedicated Screen Flows using the existing field sets.